Assignment 7

Page No.
Date : / /

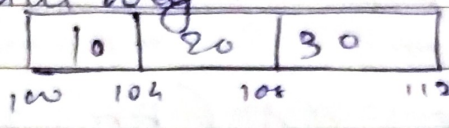
1] What is meant by array? Explain the use of array with example.

→ 1) Array is considered as a linear Data Structure
2) Derived Data type in C, C++ and Java.
3) Holds/stores 'multiple homogeneous variables in index-format'

Eg. int Arr[3] = {10, 20, 30};

Uses of Array :

1] Memory for all elements of array gets allocated in a sequential way



2] We can access all elements - using single name, so no need to remember multiple names.
ie. Fast element access.

3] Compact Data storage : Commonly used for: e.g. User records, inventory details & other sequential data.

4] Handling Buffer Data : Essential for temporary storage of data during transfer between 2 places in a system.
Eg. Reading files or handling I/O streams.

5] Implementation of Matrices
- 2D arrays are used to represent matrices which are crucial in operations.
Eg. Transformations in graphics, solving systems of linear equation & statistical analyses.

6] Image Representation: In computer graphics, images are stored as arrays of pixels where each pixel represents a colour.
- Manipulations & effects applied to images are often performed via operations on these arrays.

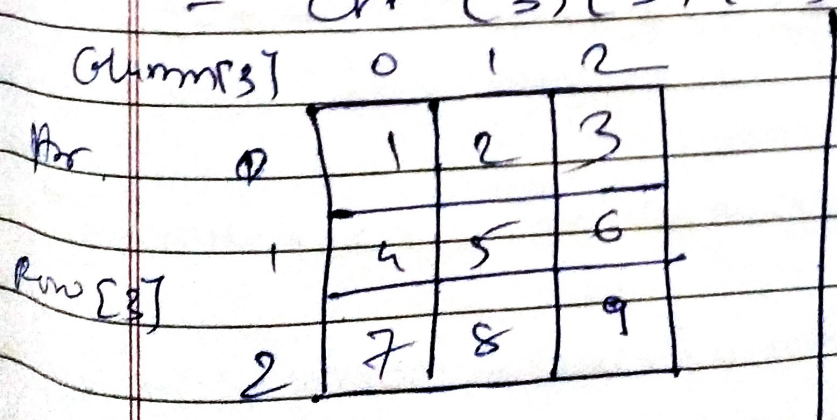
2] What are the types of array

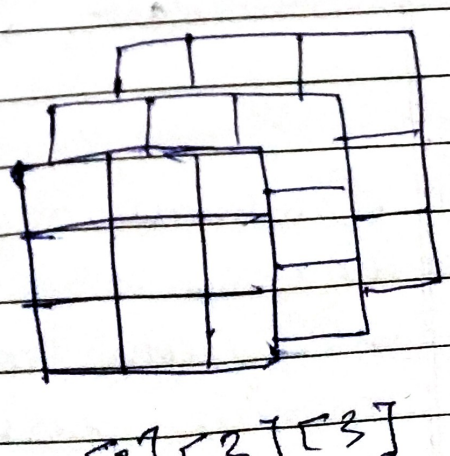
1] One-Dimensional Array:
A linear array with a single subscript, where elements are stored one after another
Arr [3] = {10, 20, 30}

	0	1	2
Arr	10	20	30
	100	104	108	112

2] Two Dimensional Array:
An array with 2 dimensions.
Eg. Arr [3][3];
Statement reading: Arr is a 2D array which contains 3 1D arrays in it, each 1D array contains 3 elements, each element is of type integer.

3] Multi-dimensional Array: [Array of arrays]
- An array with more than 2 dimensions.
- Arr [3][3][3]





4] Associative array :- [maps / dictionaries]
An abstract data type that stores data in key-value pairs.

5] Jaggered array :- Where each member array has a default value of null.

6] Array of pointers :- Whose all elements are pointers.

7] Dynamic array :- Array that automatically resizes.

8] What are different ways of initializing the elements of array?

→ 1] with size & initialization list :
- Create array by specifying 'length' of array & initializing array immediately using 'member initializer' list.

Syntax : Datatype Arrayname [length] = {v1, v2 ... vn};

int Arr [5] = {1, 2, 3, 4, 5};

2] Array initialization with length & with declaration :
- At time of creating array, if we initialize it using member initialization list then it is optional to specify length of array in [].
- Because length of array is automatically deduced by the compiler.

Syntax : Datatype Arrayname [] = {v1, ..., vn};

int Arr [] = {10, 20, 30};

3] Declaration & initialization of array with less members :
- Specify length of array with some value but initialize less no. of elements in it.

E.g. int arr[7] = {10, 20, 30, 40};

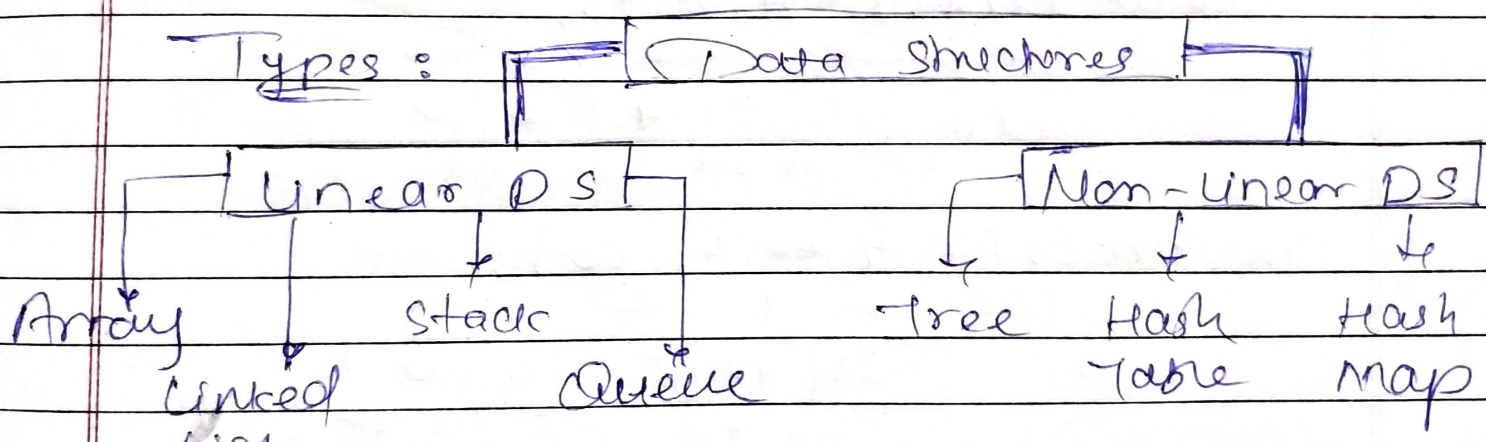
arr.	10	20	30	40	0	0	0
	1	2	3	4	5	6	7

4] Creation & Initialization using :
Member by Member Initialization
ie: Initialization after declaration.
Eg: ⇒ int Arr[3];
Arr[0] = 1;
Arr[1] = 2;
Arr[2] = 3;

4] What is meant by data structure? and what are types of data structures.

→ Data Structures:
- A way of storing & representing a data in a particular format.
- Concept of Datastructure - 'Language independent'
- Applicable in every Programming Language.

Types:



5] Explain different ways of memory allocation for an array.

→ Memory of all elements of array gets allocated in a - 'Sequential way'

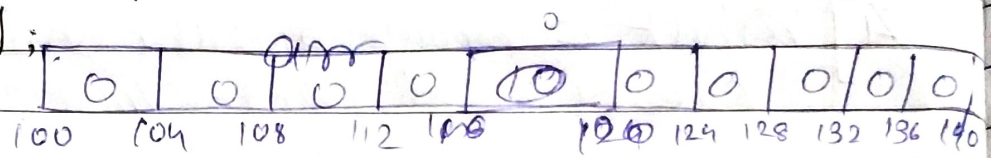
Static memory allocation:
- Compiler allocates static memory for an application at load time.
- Exact size & type of memory must be known at compile time

Date : / /

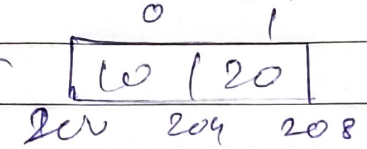
2] Dynamic memory allocation :
→ The run-time system allocates dynamic memory during run-time.
→ The exact sizes or units do not have to be known by the compiler in advance.

3] Read the below statements & draw its diagrammatic Representation

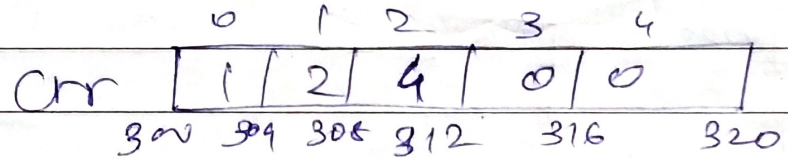
a) int arr[10];



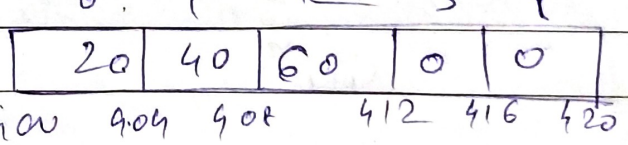
b) int brr[2] = {10, 20};



c) float crr[5] = {1, 2, 4};



d) int drr[5] = {20, 40, 60};



e) const int Demo[2] = {10, 20, 30, 40};

4] Find out problems & correct it.

a] int arr[4] = {1, 2, 3, 4, 5, 6, 7};
No of elements can't be more than length.
e. int arr[7] = {1, 2, 3, 4, 5, 6, 7}; or
int arr[4] = {1, 2, 3, 4}

Date : / /

b. int brr[];
Array length is not defined
⇒ Specify size Eg: int brr[3]

c. int crr[] = {10, 20, 30, 40};
← There is no mistake.

d. int drr[3+2] = {7+1, 3*5, 100, 10-1};
← No problem. length is 5 and elements are - 8, 15, 100, 9, 0

e. int i = 4 | int arr[i] = {23, 45, 67, 89};
length cannot be a character, it has to be a integer
ie: int arr[4] = {23, 45, 67, 89}

f. What is meant by size of operator, explain with example
⇒ It gives no of bytes allocated to specific data object (variable)
Eg. int i = 4; ∴ size of (i) = 4 bytes.
int arr[4]; ∴ size of (arr) = 16 bytes.
char c = 'A'; ∴ size of (c) = 1 byte.
double d = 50.55; ∴ size of (d) = 8 bytes

g. Predict output of below code snippet

```
int main()
{ int arr[3] = {21, 43, 50};
  int x = 0;
  x = arr[2] + 20 + arr[0];
  x++;
  printf("%d", x);
  return 0;
}
```

arr[2] = 50
arr[0] = 21
x = (50) + 20 + (21)
x = 91

x++ = 91 + 1
= 92

(07)

```
int main()
{ float arr[4] = {98.3, 4.5, 51.5, 7.5};
  int i=0;
  printf("%f :: ", arr[i]);
  i++;
  printf("%f :: ", arr[i]);
  i++;
  printf("%f", arr[i]);
  return 0;
}
```

Output : 98.300003 :: 4.500000 :: 51.500000